

Локализация в мобильном приложении

Локализация в МП — это процесс адаптации приложения под разные языки и региональные стандарты (форматы дат, валют, чисел и т. д.), чтобы пользователи из разных стран могли использовать его на родном языке. Грамотная реализация повышает удобство для пользователей и расширяет аудиторию приложения.



Правила локализации (как минимизировать распространенные проблемы перевода на этапе подготовки ключей и повысить общее качество локализации приложений):

<https://link.sbis.ru/article/fe82bcad-4349-4bb5-a329-4cfacbbc2ecb>

Статическая и динамическая локализация

Основной принцип локализации в Android работает по следующему алгоритму:

1. Android SDK проверяет язык системы (`Locale.getDefault()`).
2. Ищет папку `res/values-<язык>-<регион>` (например, `values-ru-RU`).
3. Если её нет, ищет `res/values-<язык>` (например, `values-ru`).
4. Если и её нет, использует дефолтные ресурсы из `res/values/` .

Пример:

- Пользователь с русским языком (`ru-RU`) → Android загружает строки из `res/values-ru/strings.xml` .
- Если `values-ru` нет → берёт строки из `res/values/strings.xml` (английские).

В мобильных приложениях нашей компании мы написали свою динамическую локализацию, где язык берется не с устройства, а из настроек вашей учетной записи Saby, который вы можете выбрать в разделе настроек мобильного приложения или веб версии:

ООО "Голосовой ввод"



Голосовой
Пётр Петрович



В сети



Электронная подпись



Уведомления

Вкл.



Безопасность



Тема оформления



Язык

Русский



Поддержка Saby



Saby AppMarket



Политика конфиденциальности



Выйти

Версия: 25.2118 (G-510000001)

Дата установки приложения

12.02.2025



Меню



События



Календарь



Встречи



Настройки



Финансы



Подробнее об алгоритмах смены языка в МП:

https://wi.sbis.ru/doc/platform/developmentapl/internalization/change-lang/#_3

Поддерживаемые языки в мобильных приложениях Тензора

На данный момент в наших мобильных приложениях полностью поддерживается только 3 языка:

- ru — русский;
- en — английский;
- kk — казахский.

И еще 6 в разработке, для которых локализация уже считается обязательной, но приложения еще не осуществили полную поддержку:

- el — греческий;
- es — испанский;
- fr — французский;
- tk — туркменский;
- uz — узбекский;
- zh — китайский.

Локальная сборка под любой поддерживаемый язык

Имея доступ к коду, можно локально собрать версию приложения под любой язык. Если для каких-то строк локализация будет отсутствовать, то по умолчанию используется обязательная английская локализация для этой строки.

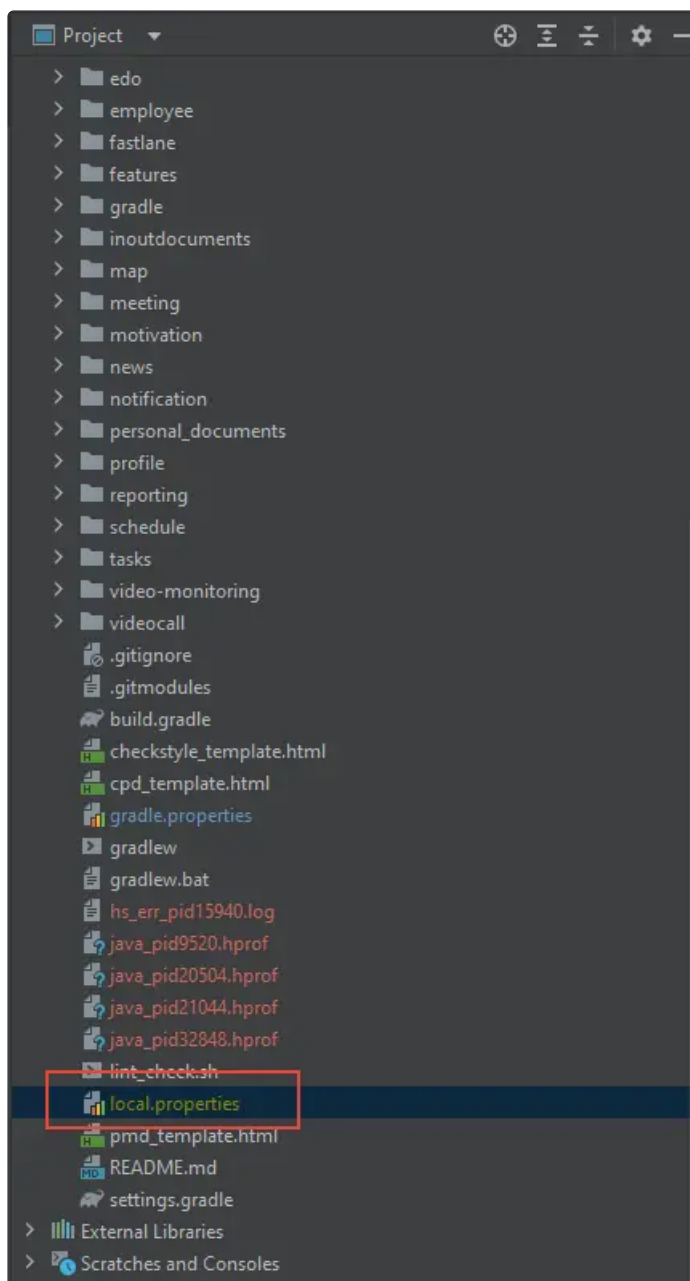
Способ 1. Расширение списка поддерживаемых языков

По умолчанию в приложении Saby доступны только 2 языка — русский и английский.

В разделе настроек приложения можно выбрать язык, в списке вам будет доступна обширная перечень, но применяться будут только вышеупомянутые русский и английский.

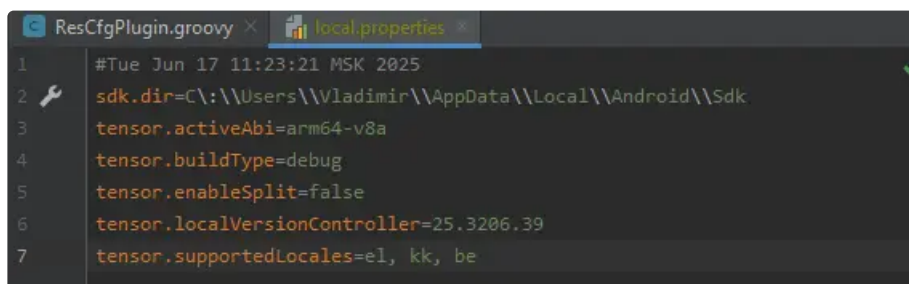


Чтобы сделать язык доступным для выбора, необходимо найти файл **local.properties** из корня проекта. Файл локально генерируется для вашего проекта и находится в исключениях для git.



С помощью ключа **tensor.supportedLocales** можно установить указать перечень поддерживаемых языков в общепринятом формате ISO, например, ru — для русского, en — для английского и тд.

Пример:



Чтобы изменения применились, необходимо синхронизировать и запустить проект.

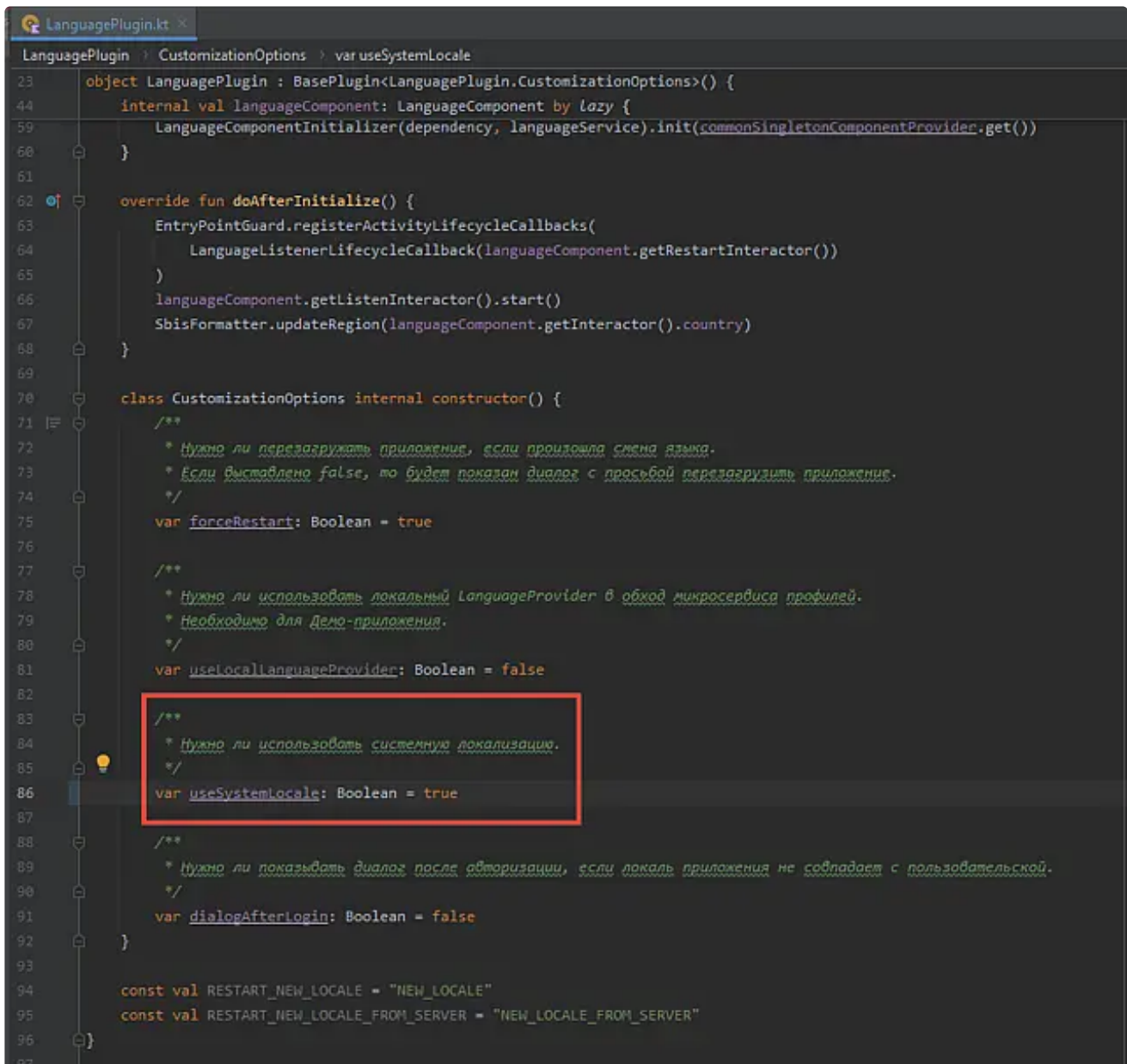
Готово. После запуска проекта вы сможете выбрать указанные языки в настройках приложения. Смена языка запустит процесс перезапуска, после которого приложение будет работать на выбранной вами локализации.

Способ 2. Принудительно использовать локализацию устройства

Данный способ позволяет приложению работать на локализации устройства, но чтобы язык сработал для приложения, его также необходимо добавить в **local.properties** из способа 1.

Далее для использования локализации устройства сделайте следующее:

1. Найдите плагин `ru.tensor.sbis.language.LanguagePlugin` из папки `auth` в модуле `language`.
2. В файле найдите `CustomizationOptions` и переключите флаг `useSystemLocale` в `true`.



```
LanguagePlugin.kt
LanguagePlugin > CustomizationOptions > var useSystemLocale

23 object LanguagePlugin : BasePlugin<LanguagePlugin.CustomizationOptions>() {
44     internal val languageComponent: LanguageComponent by lazy {
59         LanguageComponentInitializer(dependency, languageService).init(commonSingletonComponentProvider.get())
60     }
61
62     override fun doAfterInitialize() {
63         EntryPointGuard.registerActivityLifecycleCallbacks(
64             LanguageListenerLifecycleCallback(languageComponent.getRestartInteractor())
65         )
66         languageComponent.getListenInteractor().start()
67         SbisFormatter.updateRegion(languageComponent.getInteractor().country)
68     }
69
70     class CustomizationOptions internal constructor() {
71         /**
72          * Нужно ли перезагружать приложение, если произошла смена языка.
73          * Если выставлено false, то будет показан диалог с просьбой перезагрузить приложение.
74          */
75         var forceRestart: Boolean = true
76
77         /**
78          * Нужно ли использовать локальный LanguageProvider в обход микросервиса профилей.
79          * Необходимо для Демо-приложения.
80          */
81         var useLocalLanguageProvider: Boolean = false
82
83         /**
84          * Нужно ли использовать системную локализацию.
85          */
86         var useSystemLocale: Boolean = true
87
88         /**
89          * Нужно ли показывать диалог после авторизации, если локаль приложения не совпадает с пользовательской.
90          */
91         var dialogAfterLogin: Boolean = false
92     }
93
94     const val RESTART_NEW_LOCALE = "NEW_LOCALE"
95     const val RESTART_NEW_LOCALE_FROM_SERVER = "NEW_LOCALE_FROM_SERVER"
96 }
97
```

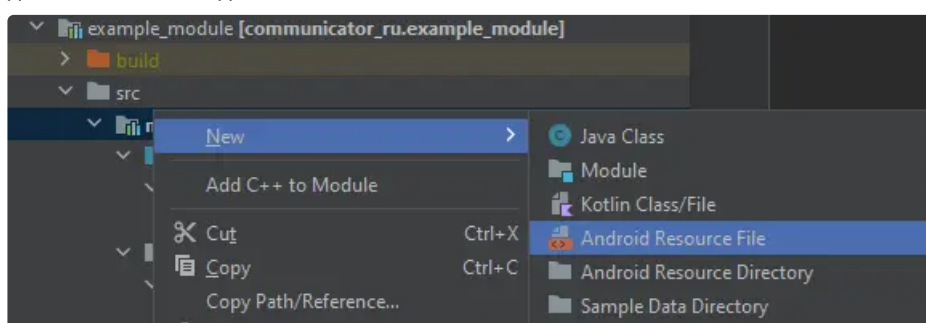
Готово. После пересборки приложения в МП будет использоваться локализация устройства, которую вы свободно можете выбрать через настройку языка вашего устройства.

Создание и использование ресурсов в Android

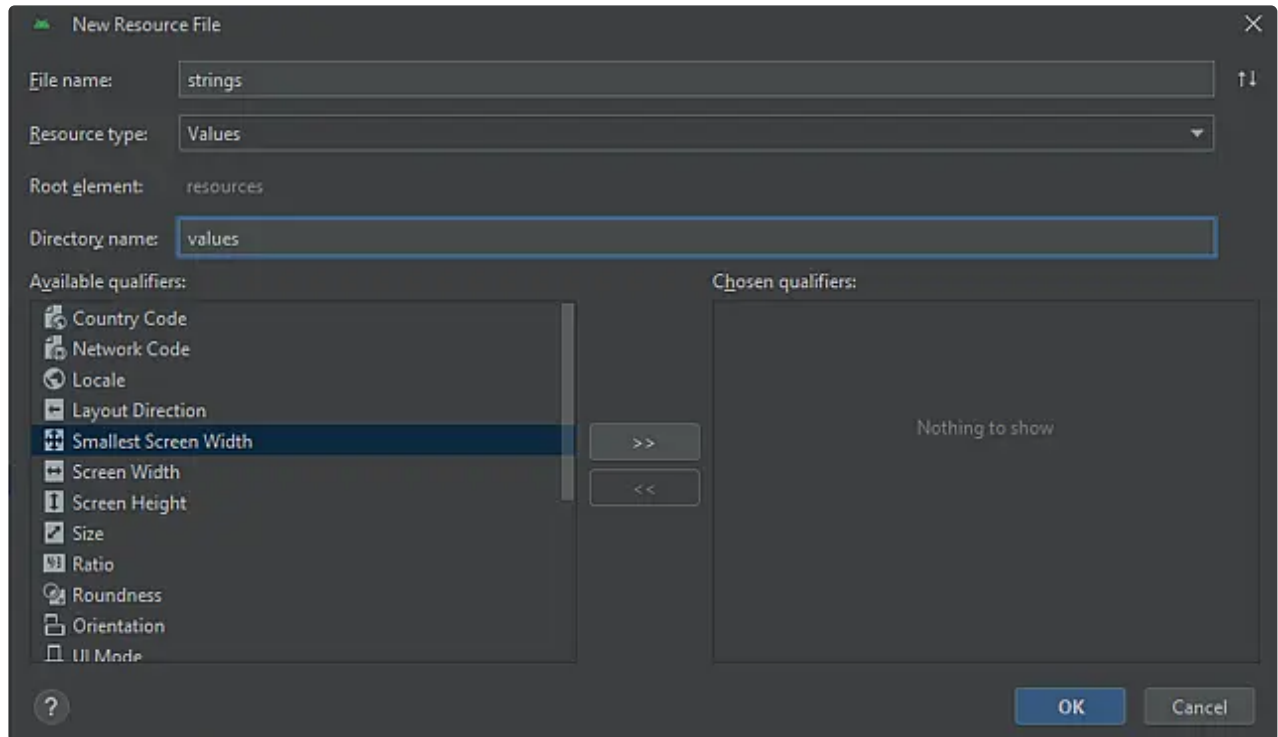
Создание ресурсов локализации

Чтобы создать ресурсы локализации под разные языки, сделайте следующее:

1. В модуле откройте создание файла "Android Resource File" от любой папки. Если ресурсы ранее отсутствовали, то далее папка `res` создастся автоматически.

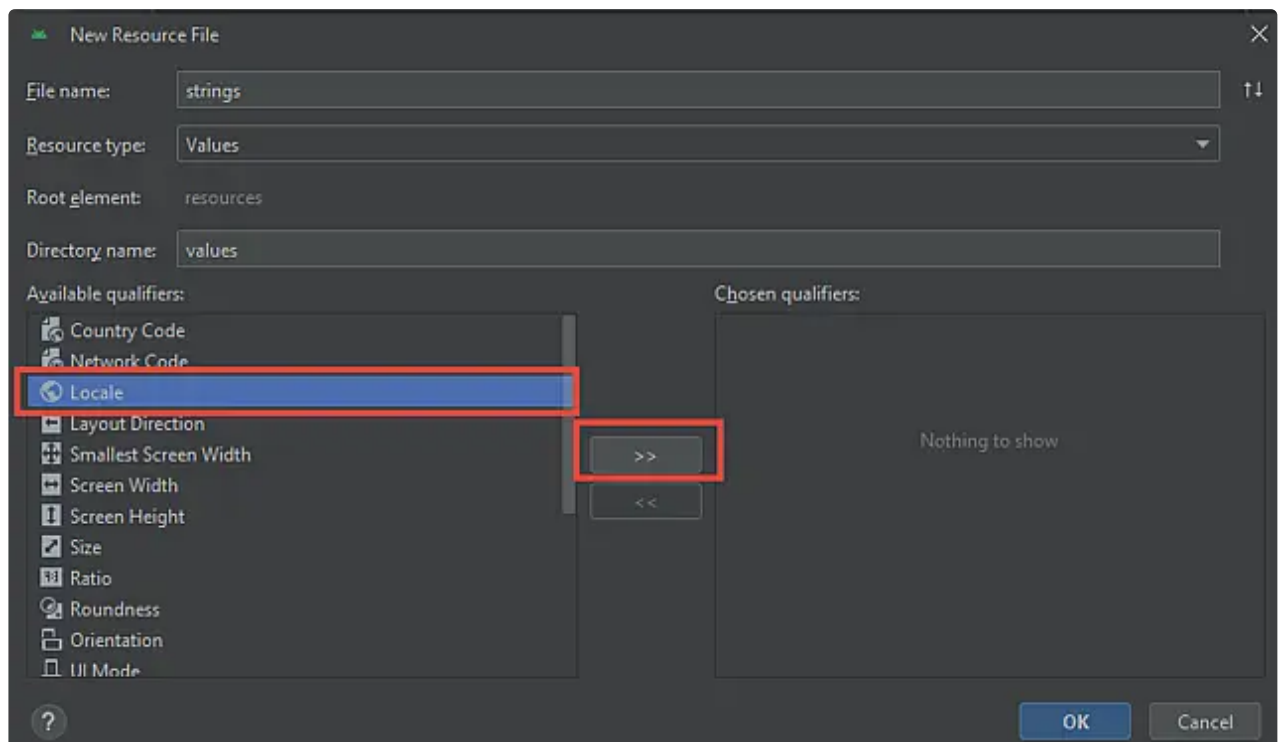


2. Название файла "strings", тип стандартный — "Values" → кнопка OK.

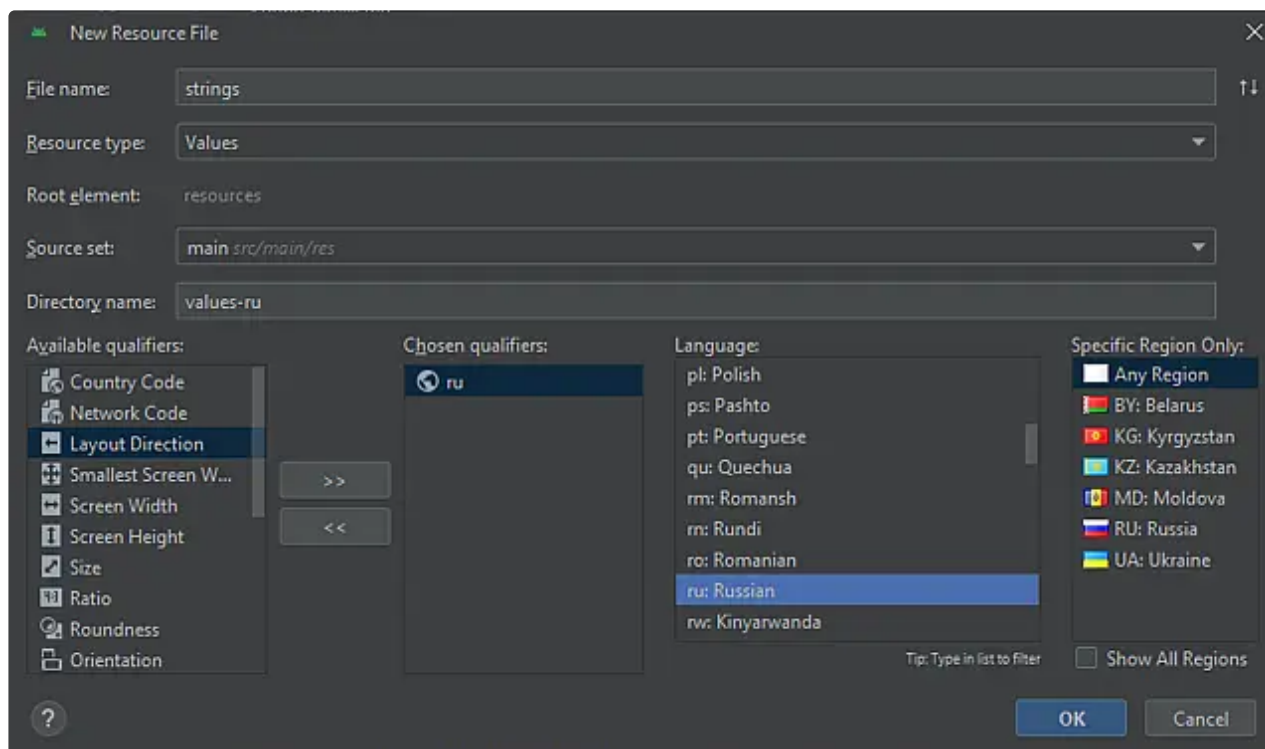


Таким образом вы создали стандартные ресурсы строк, которые будут использоваться по умолчанию. В случае вашего приложения — это английский язык.

3. Чтобы создать ресурсы локализации под другие языки, необходимо создать еще один файл strings, но с добавлением настройки локализации:



В списке Language выберите необходимый язык, directory name будет выбран автоматически, затем нажмите "OK".



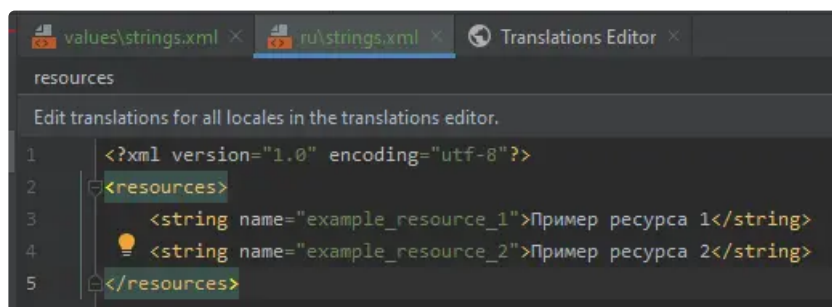
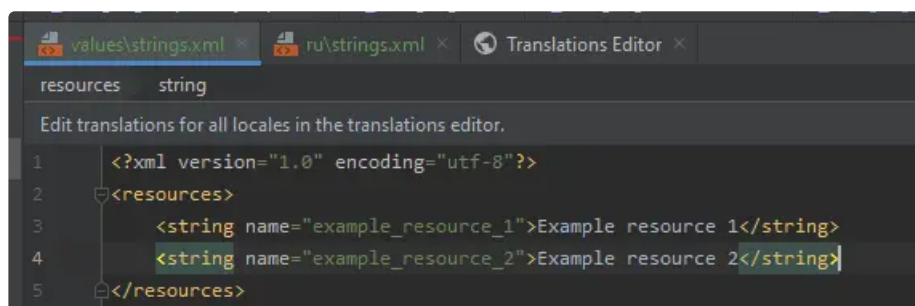
После данных действий вы увидите:

1. Появление папки **res**, если она отсутствовала в модуле.
2. Две папки **values** и **values-ru** с файлами **strings.xml**, в которых можно создавать новые строки.

Создание строк в ресурсах локализации

Для добавления новой строки необходимо открыть файл "strings" соответствующей локализации и указать необходимые вам строки в следующем формате:

```
<string name="название ресурса">"Значение"</string>
```



Теперь строки **example_resource_1** и **example_resource_2** будут иметь 2 перевода — для русского и для английского языка в зависимости от локализации приложения.

Использование строк из ресурсов

Для получения строки из ресурсов необходимо использовать метод `getString(@StringRes int id): String` на классах **Context** или **Resources**. В свою очередь ресурсы или контекст вы можете получить у **Application**, **Activity**, **Fragment**, **View**, их наследников и у многих других Android-компонентов.

```
1 package ru.tensor.sbis.example_module
```

```
2
3 import android.content.Context
4 import android.content.res.Resources
5 import ru.tensor.sbis.example_module.R
6
7 class SomeClass {
8
9     /**
10      * Получить строку напрямую из ресурсов.
11      */
12     fun getExampleString(resources: Resources): String =
13         resources.getString(R.string.example_resource_1)
14
15     /**
16      * Получить строку напрямую из контекста.
17      */
18     fun getExampleString(context: Context): String =
19         context.getString(R.string.example_resource_1)
20
21     /**
22      * Получить строку из ресурсов контекста.
23      */
24     fun getExampleString2(context: Context): String =
25         context.resources.getString(R.string.example_resource_1)
26 }
```